

# Log Sanitization and Protection

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 14: Monitoring and Auditing AI Systems

## 1. The Hidden Sensitivity of Log Data

Log files occupy an unusual position in the security landscape: they are generated to support security and operations, yet they often become a security liability in their own right. The lecture opens with this tension and it is worth dwelling on it. A developer who logs the full HTTP request body to debug an API problem may inadvertently capture every API key, session token, and credit card number that passed through the application for weeks. A database slow-query log intended to catch performance regressions may include the literal SQL queries, which contain the data values users submitted. An AI system that logs its inference inputs to support reproducibility may capture sensitive patient data, legal documents, or source code pasted by users.

The exam approach to this topic is two-pronged. First, log sanitisation: removing sensitive data from logs before they are stored. Second, log protection: securing logs that exist so that they cannot be read by unauthorised parties, altered by attackers, or lost. Both are necessary; neither alone is sufficient. An unsanitised log that is perfectly protected is still a liability. A sanitised log that is unprotected against tampering is forensically worthless.

## 2. Categories of Sensitive Data Commonly Found in Logs

| Data Category                                  | How It Ends Up in Logs                                                           | Regulatory Relevance                                              |
|------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Passwords and authentication tokens            | User errors (typing password in username field); session token logged in URL     | Breach notification laws; PCI-DSS requirement 3                   |
| API keys and secrets                           | Error messages that include full request headers; debug logging of configuration | Intellectual property; service account abuse if exposed           |
| Credit card numbers                            | Full request body logging in payment systems; debug mode left on in production   | PCI-DSS prohibits logging full PAN or CVV under any circumstances |
| Social Security and government ID numbers      | Form submission logs; identity verification workflow logging                     | HIPAA, GLBA, state privacy laws                                   |
| Personal health information (PHI)              | Healthcare AI inference input logging; clinical note processing                  | HIPAA minimum necessary rule                                      |
| Customer personal information (PII)            | User profile updates; support chat transcripts; AI chatbot prompt logs           | GDPR, CCPA, and equivalent regulations globally                   |
| Internal IP addresses and architecture details | Network diagnostic logging; error stack traces                                   | Reconnaissance value for attackers                                |
| Confidential business data                     | AI prompts containing proprietary documents or source code                       | Trade secret law; contractual confidentiality obligations         |

### 3. Sanitisation Technique 1: Pattern-Based Redaction

---

Pattern-based sanitisation uses regular expressions or specialised libraries to detect and replace sensitive data patterns automatically. This is the most widely deployed sanitisation technique because it is fast, operates inline during log collection, and covers the most common sensitive data types without needing semantic understanding of the log content.

- Credit card numbers: regex matching 13 to 16 digit sequences with appropriate spacing patterns, validated against Luhn algorithm checksums.
- Social Security numbers: nine-digit patterns in NNN-NN-NNNN or NNNNNNNNNN format.
- Email addresses: standard RFC 5322 email pattern, often replaced with a domain-preserving placeholder such as [REDACTED]@domain.com.
- API key formats: provider-specific patterns such as sk-[alphanumeric 48] for OpenAI keys or AKIA[16 alphanumeric] for AWS access key IDs.
- JWT tokens: three Base64URL-encoded sections separated by dots, matching the header.payload.signature structure.

The limitation of pattern-based sanitisation is that it is defined by what you expect. A novel API key format or an organisation-specific secret format that does not match any known pattern will pass through undetected. This is why pattern-based sanitisation is necessary but not sufficient and must be combined with additional techniques.

### 4. Sanitisation Technique 2: Keyword Blacklists and Field-Level Redaction

---

Keyword blacklists complement pattern matching by targeting context rather than format. A password value does not have a predictable format (it could be eight characters or sixty), but the word 'password' or 'passwd' appearing immediately before an equals sign or a colon is a reliable indicator that sensitive data follows.

For structured JSON logs, field-level sanitisation is more precise than text scanning. Define a set of field names that always carry sensitive data and redact their values unconditionally: password, api\_key, credit\_card, cvv, ssn, token, secret, private\_key. Keep all other fields intact. This approach is cleaner than regex on unstructured text because you never accidentally redact a customer's order number simply because it happens to be a 16-digit string.

**Design principle:** For structured logs, prefer field-level sanitisation over full-text regex scanning. It is faster, produces fewer false positives, and is easier to audit: you can list exactly which fields are always redacted.

### 5. Sanitisation Technique 3: Hashing for Privacy-Preserving Tracking

---

Complete redaction removes the sensitive value and replaces it with a placeholder. This is appropriate for data you never need to see again (CVV codes, passwords). But some sensitive values need to be tracked across log entries without being exposed: user IDs, session identifiers, device fingerprints.

Hashing solves this problem by transforming the sensitive value into a fixed-length digest that is consistent (the same input always produces the same hash) but not reversible (you cannot recover the

original value from the hash alone). Using a salted hash with a secret salt makes the output unguessable even to someone who has access to the log database but not the salt.

| Approach                         | Reversible       | Consistent (trackable) | Brute-force resistant            | Use case                                                       |
|----------------------------------|------------------|------------------------|----------------------------------|----------------------------------------------------------------|
| Full redaction (placeholder)     | No               | No                     | N/A                              | Values never needed again: CVV, passwords                      |
| Unsalted hash (SHA-256)          | No               | Yes                    | No (rainbow tables work)         | Low-sensitivity IDs where tracking matters                     |
| Salted hash (HMAC-SHA256)        | No               | Yes (with same salt)   | Yes                              | User IDs, session tokens needing cross-log tracking            |
| Pseudonymisation (token + table) | Yes (with table) | Yes                    | Yes (table protected separately) | Cases where re-identification may be needed for investigations |

## 6. Sanitise at Collection Time, Not Retrospectively

The lecture makes an important sequencing point: sanitise logs as they are generated or collected, before they reach any storage system. This is the only approach that guarantees sensitive data is never stored. Retrospective sanitisation, scanning stored logs and redacting sensitive content after the fact, has two fundamental weaknesses.

- The data already exists in storage between the time it was written and the time the sanitisation job runs. Any breach or unauthorised access during that window exposes the sensitive data.
- Retrospective sanitisation may miss data that was stored in backup snapshots, log shipping destinations, or SIEM ingestion pipelines that ran before the sanitisation job completed.

Build sanitisation into the logging library or log collection agent so that it executes in the same thread as log generation, before any network transmission or disk write. This is the defence-in-depth principle applied to the data lifecycle: protect data at its origin, not only at its destination.

## 7. Real-World Incident: Uber (2022 Breach via Exposed Credential in Code)

**Real-World Incident: Uber (2022):** In September 2022, a threat actor gained access to Uber's internal network by compromising an employee's VPN credentials through social engineering. Once inside, the attacker found a PowerShell script in an internal network share that contained hardcoded credentials for Uber's Privileged Access Management (PAM) system. These credentials provided access to AWS, Google Cloud, and other internal tools. While this was not strictly a log-based exposure, the incident illustrates the precise risk that log sanitisation addresses: credentials embedded in internal artefacts (scripts, configuration files, and by extension log files) become a lateral movement enabler for any attacker who achieves initial access. Had those credentials been in a log file rather than a script, the outcome would have been identical. Organisations that log full request bodies, configuration states, or debug output without

sanitisation create the same risk.

For the SecAI+ exam, connect this incident to the principle that any internal artefact containing unsanitised credentials or secrets is a security liability equivalent to storing passwords in plaintext. Log files are internal artefacts and must be treated accordingly.

## 8. Log Protection: Encryption, Access Control, and Integrity

---

Sanitisation reduces what is sensitive in logs. Protection secures what remains. The lecture addresses three distinct protection dimensions:

- **Encryption at rest:** Encrypt log storage using AES-256 for file-based logs, Transparent Data Encryption (TDE) for database-backed log storage, and encrypted storage buckets in cloud environments. Even sanitised logs reveal system architecture, user behaviour patterns, and operational details that are valuable to attackers.
- **Access control:** Implement role-based access control (RBAC) on log storage. Developers access application logs; security analysts access security event logs; payment system logs are restricted to finance security teams. Overly permissive log access means that any account compromise exposes all logs. Follow the principle of least privilege.
- **Integrity protection:** Prevent tampering and deletion. Attackers routinely delete logs to cover their tracks after a breach. Use append-only log storage where entries cannot be modified or deleted. Implement cryptographic log signing where each entry includes a hash chained to the previous entry. Forward logs immediately to an immutable external destination such as a SIEM or write-once cloud bucket.

## 9. Secure Log Transmission

---

Logs travel across networks from applications to log collectors to central storage. Each hop is an interception opportunity. The lecture emphasises that log transmission must be encrypted using TLS, with authenticated endpoints so that only legitimate log collectors can receive data. Specific risks from unencrypted log transmission include:

- **Network sniffing:** an attacker with access to the network segment between the application server and log collector can capture all log traffic in plaintext.
- **Man-in-the-middle injection:** an attacker who can intercept and modify log traffic can inject or delete log entries, undermining forensic integrity.
- **Credential exposure:** if any unsanitised credentials appear in logs, unencrypted transmission immediately exposes them to any network observer.

Authenticated log forwarding, where the receiving collector verifies the identity of the sending application, prevents rogue systems from injecting false log entries into the central log repository. This is particularly important in environments where attackers may try to flood the SIEM with false data to obscure a real attack.

## 10. Retention, Disposal, and Regulatory Compliance Alignment

---

The lecture closes with the full log lifecycle: creation, sanitisation, transmission, storage, retention, and secure deletion. The retention and disposal phases are often neglected.

| Regulation           | Key Log-Related Requirement                                                         | Implication for Sanitisation and Retention                                                     |
|----------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| GDPR (EU)            | Personal data must be minimised, purpose-limited, and not retained beyond necessity | Sanitise PII from logs; enforce retention limits; prove deletion on request                    |
| PCI-DSS              | Full PAN and CVV must never be logged; access logs must be retained 12 months       | Pattern-based redaction of card data at collection; 12-month access log retention              |
| HIPAA                | PHI must be protected; minimum necessary standard applies                           | Sanitise PHI from AI inference logs; restrict access to log storage to minimum necessary roles |
| SOX (Sarbanes-Oxley) | Financial system logs must be retained 7 years                                      | Long-term archival with integrity protection; encrypted immutable storage                      |
| CCPA (California)    | Consumers can request deletion of personal data                                     | Log systems must support search and deletion of PII by individual; pseudonymisation aids this  |

When exam questions ask about compliance requirements for log handling, identify the regulation first, then apply the specific requirement. The most commonly tested point is PCI-DSS: under no circumstances may full credit card numbers (PANs) or CVV codes be stored in logs, regardless of encryption or access controls.

## Key Terms Glossary

| Term                              | Definition                                                                                                                                                     |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Log sanitisation                  | The process of removing or obfuscating sensitive data from log entries before they are stored, to reduce the risk of data exposure from log breaches.          |
| Pattern-based redaction           | Using regular expressions or format-matching algorithms to detect and replace sensitive data patterns such as credit card numbers or API keys in log text.     |
| Keyword blacklist                 | A list of field names or contextual terms whose associated values are always redacted from logs, regardless of the value's format.                             |
| Salted hash                       | A hash produced by combining the input value with a secret salt before hashing, making the output resistant to rainbow table attacks and brute-force reversal. |
| Append-only log storage           | Log infrastructure that accepts new entries but prevents modification or deletion of existing entries, preserving forensic integrity.                          |
| Transparent Data Encryption (TDE) | Database-level encryption where the storage engine automatically encrypts data as it is written to disk and decrypts it for authorised queries.                |

|                              |                                                                                                                                                            |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Log integrity                | The property that log entries have not been altered or deleted since they were originally written, often proved through cryptographic chaining or signing. |
| Retrospective sanitisation   | Scanning and redacting sensitive data from logs after they have been stored, as opposed to sanitising at collection time; considered a weaker approach.    |
| PAN (Primary Account Number) | The full credit card number; PCI-DSS explicitly prohibits logging PANs or CVV codes under any circumstances.                                               |
| Log shipping                 | The process of forwarding log entries from their source to a centralised storage or analysis system, which must use encrypted and authenticated channels.  |

## Quick Revision Checklist

---

- Log sanitisation removes sensitive data before storage; log protection secures what remains. Both are required.
- Common sensitive data in logs: passwords, API keys, credit card numbers, SSNs, PHI, PII, internal IPs, and confidential business data.
- Three sanitisation techniques: pattern-based redaction (regex), keyword blacklists and field-level redaction, and hashing for privacy-preserving tracking.
- Sanitise at collection time, not retrospectively: data that is never stored cannot be breached from storage.
- For structured JSON logs, field-level sanitisation is more precise and auditable than full-text regex scanning.
- Salted hashes enable tracking of sensitive identifiers across log entries without storing the actual values.
- Log protection requires encryption at rest (AES-256 or TDE), role-based access control, and integrity protection (append-only storage or cryptographic signing).
- Log transmission must use TLS with authenticated endpoints to prevent sniffing and injection.
- PCI-DSS: PANs and CVV codes must never appear in logs under any circumstances, even encrypted.
- Secure disposal at end of retention period is as important as protection during storage; overwrite or use certified deletion tools.

## 10 Exam-Based MCQs: Log Sanitization and Protection

---

*Test yourself after reading. These questions are written in real exam style, scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.*

**Q1. Scenario:** A financial technology company processes credit card transactions for online retailers. The company recently engaged a Qualified Security Assessor (QSA) for their annual PCI-DSS assessment. During the assessment, the QSA runs a log scan tool against the application server log directory. A PCI-DSS QSA auditing a payment processing company discovers that the application server logs contain full 16-digit credit card numbers in several request body logs. The development team explains that the

logging was enabled during a debugging session six months ago and never turned off. Which remediation step is MOST urgent according to PCI-DSS requirements?

- A) Encrypt the log files using AES-256 and restrict access to the security team
- B) Immediately remove or securely delete the log files containing PANs, then implement pattern-based sanitisation at collection to prevent recurrence
- C) Set a 90-day retention limit on all log files going forward
- D) Move the log files to a separate PCI-scoped network segment

**Answer: B** **Explanation:** B is correct because PCI-DSS Requirement 3.3 explicitly prohibits storing sensitive authentication data including full PANs after authorisation. The existing logs containing PANs are in violation and must be removed or securely deleted immediately. Implementing sanitisation prevents recurrence. A is wrong because encrypting logs that contain PANs does not bring the organisation into compliance; PCI-DSS prohibits storage of PANs regardless of encryption. C is wrong because setting a retention limit does not address the logs that currently exist and are in violation. D is wrong because moving the files to a different network segment does not remove the PAN data from storage; it merely changes where the violation exists.

**Q2. Scenario:** A large enterprise has deployed an internal AI assistant for the HR department. The HR team uses the assistant to answer questions about company policies, employee records, and payroll. The security team is designing the logging architecture prior to go-live. A security engineer is designing the log sanitisation pipeline for an AI assistant that processes employee HR queries, which may include salary information, performance review notes, and disciplinary records. The engineer must ensure that cross-session analysis of employee interaction patterns is still possible. Which sanitisation approach BEST meets both requirements?

- A) Full redaction of all employee identifiers with random placeholders, discarding the mapping
- B) Complete logging of all input text with AES-256 encryption and restricted access
- C) Salted HMAC hashing of employee identifiers while redacting the text content of HR data fields
- D) Disabling input logging entirely for HR-related queries

**Answer: C** **Explanation:** C is correct because salted HMAC hashing of the employee identifier allows cross-session tracking (the same employee always produces the same hash) without storing the actual identifier. Redacting the HR text content removes the sensitive information while preserving the metadata needed for pattern analysis. A is wrong because random placeholders with no mapping table prevent consistent tracking across sessions, which the requirement specifies must be possible. B is wrong because full logging of HR text including salary and disciplinary information retains the sensitive data even under encryption, creating a liability and likely a compliance violation. D is wrong because disabling input logging eliminates the ability to detect misuse patterns or investigate incidents involving HR data.

**Q3. Scenario:** A software company runs a microservices architecture where each service logs its activities to a shared log directory on the server. The application services make outbound calls to cloud APIs for payment processing, email delivery, and analytics. During a red team exercise, the team discovers that a threat actor who gains read access to the application log directory can extract valid API keys for three cloud services from the past 30 days of logs. The keys appear because error messages logged the full



request header when external API calls failed. Which PRIMARY control would have prevented this exposure?

- A) Encrypting the log files at rest
- B) Implementing field-level or pattern-based sanitisation at log collection to redact API key formats from log messages
- C) Setting a 7-day log retention period
- D) Restricting read access to the log directory to the security team only

**Answer: B** **Explanation:** B is correct because the root cause is that API keys were written to log storage in cleartext. Sanitisation at collection time, either through field-level redaction of header values or pattern matching on known API key formats, would have prevented the keys from ever reaching storage. A is wrong because encryption at rest protects against breaches of the storage medium (disk theft, cloud bucket misconfiguration) but does not help if the attacker has already been granted read access to the log directory, as described in the scenario. C is wrong because reducing retention to 7 days limits the historical window but does not prevent future keys from being logged. D is wrong because access restriction is a compensating control that reduces the blast radius but does not address the root cause; if access control is ever misconfigured or bypassed, the keys remain in the logs.

**Q4. Scenario:** A European online service allows users to interact with an AI recommendation engine. The privacy team has received a request from a data protection authority asking the organisation to demonstrate how it handles individual erasure requests for data held in log systems. A GDPR compliance officer asks the security team to ensure that individuals can exercise their right to erasure (right to be forgotten) for data held in the log system. Which log sanitisation approach MOST directly supports this capability?

- A) Full anonymisation, replacing identifiers with random tokens and discarding the mapping
- B) Pattern-based redaction of all personal data at collection time
- C) Pseudonymisation with a secure, separately managed token-to-identity mapping table
- D) Hashing all personal identifiers with a public hash algorithm

**Answer: C** **Explanation:** C is correct because the right to erasure requires the ability to identify and delete all data linked to a specific individual. Pseudonymisation with a managed mapping table allows the organisation to look up all tokens associated with a given individual and then erase those tokens from the mapping, effectively severing the link between the log data and the person's identity without having to delete the log entries themselves. A is wrong because full anonymisation with no mapping table makes it impossible to identify which log entries belong to a specific individual, so erasure cannot be targeted. B is wrong because pattern-based redaction at collection time has already removed PII before storage, which means the data may already satisfy erasure requirements but does not provide a mechanism to respond to individual erasure requests for data that was stored before the redaction policy was implemented. D is wrong because hashing with a public algorithm (no salt) is vulnerable to rainbow table attacks, does not constitute pseudonymisation under GDPR, and does not provide a mechanism for targeted erasure.

**Q5. Which of the following BEST describes the security advantage of append-only log storage compared to standard read-write log storage?**



- A) Append-only storage automatically encrypts new log entries as they are written
- B) Append-only storage prevents attackers from deleting or modifying log entries to cover their tracks after a breach
- C) Append-only storage reduces log volume by deduplicating identical entries
- D) Append-only storage eliminates the need for log sanitisation because entries cannot be modified

**Answer: B** **Explanation:** B is correct because the defining security property of append-only storage is that existing entries cannot be modified or deleted. Post-breach log deletion is a standard attacker technique for impeding forensic investigation; append-only storage makes this impossible. A is wrong because append-only storage is a write-permission control; it does not inherently include encryption. Encryption is a separate control applied at the storage layer. C is wrong because append-only storage does not deduplicate entries; it simply prevents deletion and modification. D is wrong because append-only storage addresses integrity, not content sensitivity. Even in append-only storage, sensitive data written to a log entry remains in that entry permanently. Sanitisation at collection time is still required.

**Q6. A security architect is reviewing the log forwarding configuration for an AI platform. Logs are currently forwarded from application servers to a central SIEM over an internal network using an unencrypted syslog connection. Which risk does this configuration MOST directly create?**

- A) Log entries may be lost if the SIEM is temporarily unavailable
- B) An attacker with access to the internal network segment can read log contents in transit and potentially inject or delete log entries
- C) The SIEM may be unable to parse unencrypted syslog format
- D) Unencrypted log forwarding increases storage costs at the SIEM

**Answer: B** **Explanation:** B is correct because unencrypted transmission exposes log contents to any attacker who can observe or intercept traffic on the internal network segment. This creates two risks: confidentiality (reading sensitive log content) and integrity (injecting false log entries or deleting real ones by modifying the stream). A is wrong because SIEM availability is independent of whether the transport is encrypted; it is a redundancy and reliability concern, not a security risk of unencrypted transport. C is wrong because SIEM platforms support both encrypted and unencrypted syslog; the format is not the issue. D is wrong because encryption has no meaningful impact on SIEM storage costs; log volume is determined by content, not transport encryption.

**Q7. An organisation implements log sanitisation by running a nightly batch job that scans the previous day's stored logs and redacts any detected sensitive patterns. Which weakness does this approach have compared to sanitisation at collection time?**

- A) Batch sanitisation is less accurate than collection-time sanitisation for detecting sensitive patterns
- B) Sensitive data exists in storage for up to 24 hours between when it is written and when it is redacted, creating a window of exposure
- C) Batch sanitisation cannot be used for structured JSON log formats
- D) Nightly batch jobs consume more CPU resources than collection-time sanitisation

**Answer: B** **Explanation:** B is correct because retrospective sanitisation always leaves a window between when sensitive data is written to storage and when it is redacted. Any breach, unauthorised access, backup snapshot, or log-shipping event that occurs during that window captures the unredacted data. Collection-time sanitisation eliminates this window by ensuring sensitive data never reaches storage. A is wrong because accuracy of pattern detection is the same whether the sanitisation logic runs at collection or in a batch; the algorithms are identical. C is wrong because batch sanitisation can process structured JSON logs just as collection-time sanitisation can. D is wrong because while batch jobs do consume resources, CPU cost is not the primary security weakness of this approach.

**Q8. Which of the following correctly describes the relationship between HIPAA's minimum necessary standard and AI system log design?**

- A) HIPAA's minimum necessary standard requires that all logs containing PHI be retained for a minimum of six years
- B) HIPAA's minimum necessary standard requires that PHI in AI logs be limited to what is necessary for the specific logging purpose, and access to those logs be restricted to only those roles that need it
- C) HIPAA's minimum necessary standard prohibits logging any PHI under any circumstances
- D) HIPAA's minimum necessary standard applies only to medical device logs and not to AI inference logs

**Answer: B** **Explanation:** B is correct because HIPAA's minimum necessary standard requires covered entities to make reasonable efforts to limit the use and disclosure of PHI to the minimum necessary to accomplish the intended purpose. Applied to AI logs, this means: log only the PHI fields required for the specific monitoring purpose (not full clinical notes if only the diagnosis code is needed), and restrict access to the logs to roles that require PHI for their job function. A is wrong because the six-year retention requirement in HIPAA applies to the retention of the Notice of Privacy Practices and certain policy documents, not specifically to AI inference logs; log retention under HIPAA is generally based on state law and organisational policy. C is wrong because HIPAA does not prohibit all logging of PHI; it requires that such logging be minimised and properly protected. D is wrong because HIPAA applies to all systems processing PHI at covered entities and their business associates, including AI inference systems and their associated logs.

**Q9. A security team wants to verify that their log sanitisation pipeline is correctly redacting API keys before log storage. Which testing approach MOST effectively validates this?**

- A) Review the sanitisation code manually and verify that the regex patterns are syntactically correct
- B) Intentionally include known-format test API keys in log-generating test transactions and verify that the keys are replaced by redaction placeholders in the stored logs
- C) Check that the log storage encryption key is properly rotated on schedule
- D) Review access control lists on the log directory to confirm only authorised users have read access

**Answer: B** **Explanation:** B is correct because the only way to validate that sanitisation is working end-to-end is to put known sensitive data into the pipeline and verify it does not appear in the output. Testing with

real-format test keys (not production keys) exercises the complete sanitisation pipeline from generation through storage and confirms that the detection and redaction logic functions as designed in the actual runtime environment. A is wrong because code review can confirm that the regex patterns are syntactically valid but cannot detect environment-specific failures such as the sanitisation library being misconfigured, the wrong version being deployed, or sanitisation being bypassed for certain code paths. C is wrong because encryption key rotation is a key management control and has no bearing on whether sensitive data is being redacted. D is wrong because access controls limit who can read the logs but do not validate whether sanitisation is correctly removing sensitive content.

**Q10. Scenario:** *A financial services company operates an AI-powered anomaly detection platform for internal security monitoring. The company's data retention policy states that operational monitoring logs are retained for 90 days. The IT team implemented a retention policy in the log management tool that flags logs as expired after 90 days. An AI security platform stores logs containing behavioural data for 90 days for operational monitoring. At 90 days, logs are marked as expired in the retention system but the underlying storage is not overwritten. Six months after expiry, a threat actor breaches the log server and recovers two years of log data. Which log lifecycle control was MISSING?*

- A) Log encryption at rest
- B) Role-based access control on the log directory
- C) Secure deletion of logs at the end of their retention period
- D) Append-only log storage to prevent tampering

**Answer: C** **Explanation:** C is correct because the scenario describes logs that were logically marked as expired but not physically deleted from storage. Secure deletion, which involves overwriting the storage blocks or using certified deletion tools, would have ensured that expired logs could not be recovered after their retention period ended. A is wrong because log encryption at rest protects against storage-level breaches while the data is within its retention period; it does not address the fact that expired data was never actually deleted. B is wrong because access controls are relevant during the active retention period; once data is past its retention period and should have been deleted, access controls are irrelevant to a breach that happens after the expiry date because the data should not exist. D is wrong because append-only storage prevents modification and deletion during the active retention period, but it is not a deletion mechanism for data that has exceeded its retention period.